Unstable Bluff Detection System

Dominic Ricci

Ryder Rocha

Dylan Hancock

Alex Hardy

Ryan Kotas

Project Description

Task:

The Unstable Bluff Detection System is a system to detect changes in the bluff holding up Del Mar, California's train tracks. If the bluff appears to be shifting or falling apart, the system will send an alarm to the proper authorities. The system serves as a warning system for potentially hazardous conditions, which could endanger passengers and cause damage to the railway.

Logistics:

The system works via 6 fixed cameras placed along the bluff. The cameras are given power and a Wi-Fi connection to a central server. The server contains imaging software and a database to store images up to a month old. As well as other images, once per day, up to a year old. According to the project description, the imaging software and database are externally handled.

Design:

For our part of the ecosystem, we designed two systems: The camera system and an interface system for the server. The camera system is present on each of the 6 cameras. It determines how the camera operates. It also sends information to the interface system. The interface system communicates with the already-handled imaging software and database.

Camera System

Externally, a time interval is set. Then, every time interval, the camera will take a photo. The camera will also collect a timestamp from the internet and its geo-locational coordinates from GPS. Then, it will send the image over the network to our server-side interface. As per the project description, the timestamp and locational data are stored in the image.

Server-Side Interface System

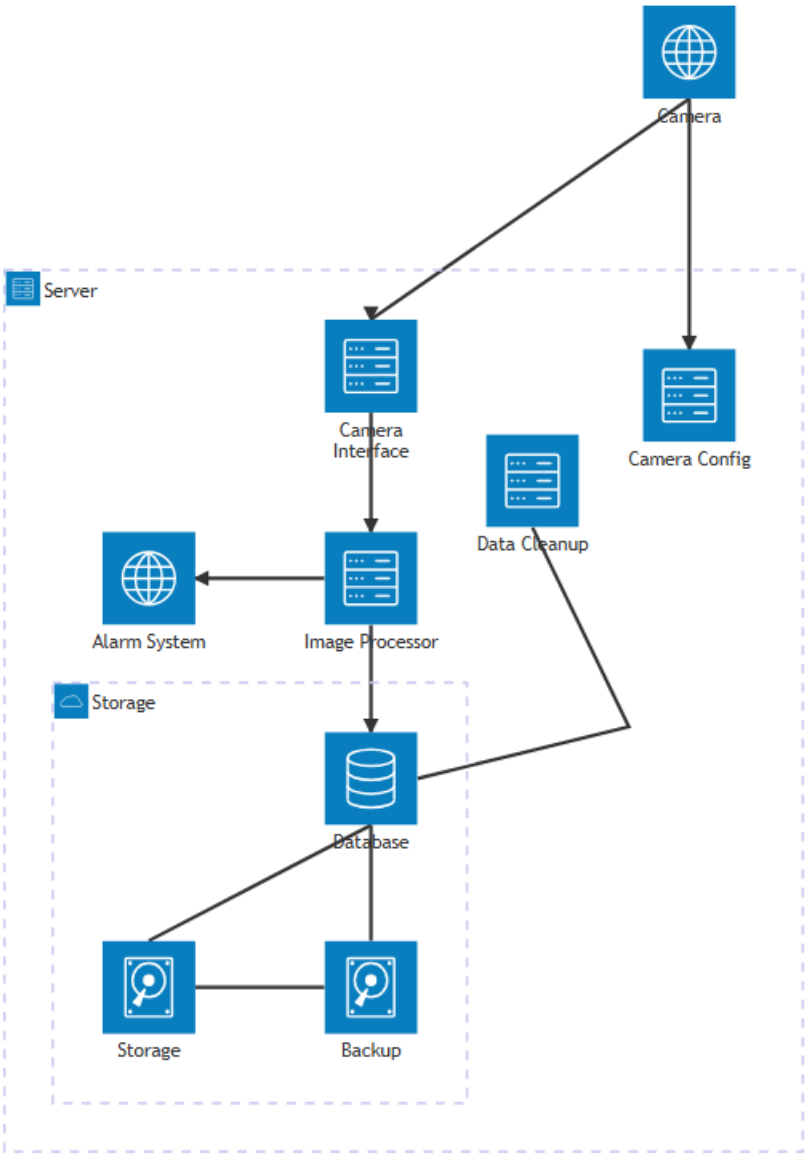
On the server side, we have three components: CameraListener, CameraConfig, and AlarmSystem. CameraListener waits for the camera to send image data. From there, the listener sends the data to the image software. The image software will then store the image in its database and determine whether it should send information about the image to the AlarmSystem. The AlarmSystem simply broadcasts information about the image to the authorities.

The CameraConfig determines the camera's resolution, time interval, and whether it takes pictures in color or greyscale. When a change is manually made to the config, it sends the updated information over the network to the camera. The camera then updates its local config based on this information.

Software Architecture Overview

For our architecture designs, we have a general software architecture design and a UML class design created in Visual Studio Code.

Software Architectural Diagram



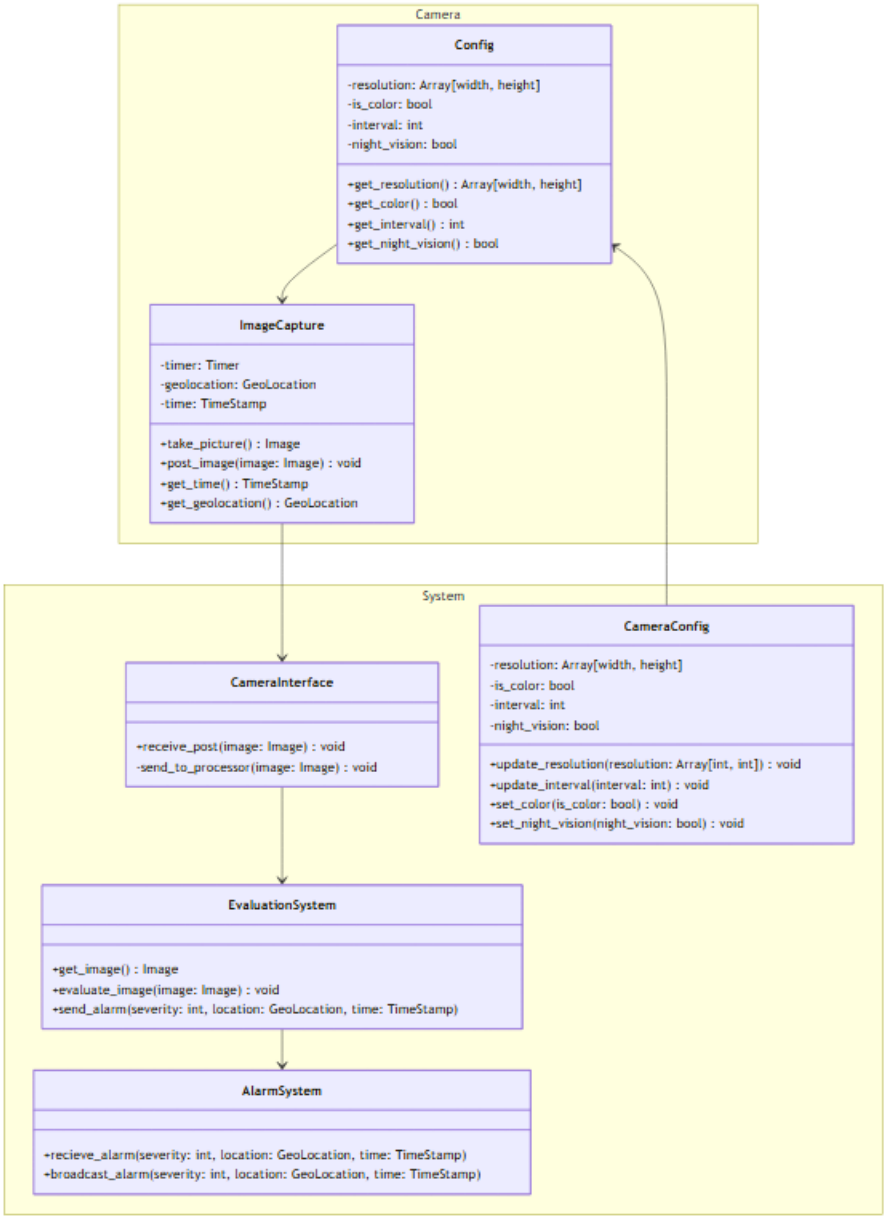
Software Architectural Description

This software architectural design has a central server that runs with a camera interface, camera configuration, the Image processor, and alarm system. The software system sends over server

post to the local database to store the images transmitted by the cameras. The image processor in the server will send an alert to the alarm system that notifies local authorities and Amtrak operators. The data cleanup all works locally to maintain essential images provided from the design specifications. The sections are with the cameras operating outside the server, and the storage of the database being located within the server at the central processing/testing center.

(PUT MORE DETAILS FOR IT)

UML Class Diagram



Description of classes

- CameraInterface: A HTTP server that handles getting the image.
- CameraConfig: Class that handles the configuration of camera settings.
- Config: Class that receives that configurations of the camera settings
- EvaluationSystem: Class that evaluates the image on a scale of 0 to 5 with an image processing algorithm, and sends out an alarm if an image receives a level 5 rating.
- AlarmSystem: Class that signals the alarm in the event of a level 5 rating.
  ImageCapture: Class that captures the image and sends over the server to the CameraInterface.

Description of attributes

- resolution: a 2D integer Array configured by width by height (i.e. [width, height]) to store the value of resolution per image. Operator has choice to choose any desirable resolution to be set in array.
- is_color: a boolean value that checks if image is configured to be in-color or in black-and-white; if true, camera will take images in-color; if false, camera will capture images in black-and-white; value assumed to be default to true.
- night_vision: a boolean value that is set to true or false if the operator chooses to set the images to be captured with night vision; value assumed to be defaulted to false.
- interval: an integer value for the time interval between image capturing per camera; measurement assumption to be in seconds.
- timer: it will pull the updated interval to then update the time.
- geolocation: a GeoLocation provided from an image's 10-megapixels; in format of 32-bit geo location.
- time: a TimeStamp provided from an image's 10-megapixels; in format of 32-bit time stamp.

Description of operations

- get_resolution: a getter method that retrieves the resolution set to a camera and returns the array.
- get_color: a getter method that retrieves the boolean value of a camera regarding the color graphics that is either set to true or false and returns said value.
- get_interval: a getter method that returns the integer interval.
- get_night_vision: a getter method that returns the boolean value of a camera using night vision and returns said value.
- take_picture: the camera captures image when prompted after set interval is completed; returns image.

- post_image: the image is sent over through a server post to the CameraInterface; returns void.
- get_time: this getter method returns a TimeStamp from the image's bits.
- get_geolocation: this getter method returns a GeoLocation from the image's bits.
- get_image: the image is retrieved from the database and returned.
- evaluate_image: the image is retrieved to then be evaluated by the processing algorithm; returns void.
- send_alarm: in the case that an image receives a level of a four or five, the image's rating, geolocation, and timestamp is retrieved and sent to the AlarmSystem.
- receive_alarm: the alarm sent from the EvaluationSystem receives an image's severity rating, geolocation, and timestamp to be recorded in the system.
- broadcast_alarm operation: it receives the values sent by the evaluation system consisting of the level integer value indicated by the image processor algorithm, the 32-bit geo location, and the timestamp from the 10 Megapixels from an image. The image set off a rating of level four or five, indicating a major change or collapse has occurred. The alarm is then broadcasted out of the server towards the Amtrak railroad operators and local lifeguards to immediately stop train traffic and evacuate all human life from the location provided by the geolocation.
- update_resolution: it receives the requested resolution prompted by the operator and sets the given camera's resolution to new array value; returns void.
- update_interval: operator provides the new time interval for camera and the integer attribute interval is set to new value; returns void.
- set_color: uses boolean attribute is_color to alter a camera's image capturing mode by the operator's choice; returns void.
- set_night_vision: uses the boolean attribute night_vision to alter a camera's night vision mode by operator's choice; returns void.
- receive_post: image from ImageCapture is received by the CameraInterface; returns void.
- send_to_processor: image sent from ImageCapture is then sent to the processing algorithm; returns void.

Development Plan:

Group Agreement:

- Work will be divided by tasks. Each person will agree to complete a task and is the main worker for that task. But, if help is needed, they can ask for assistance.

- We will communicate primarily on Discord and through GitHub projects. Expect a response within 24 hours.
- Deadlines will be set when tasks are created.
- If someone cannot complete their work, let everyone know BEFORE the deadline so that others can assist. This ensures that the work gets done even if one person can't do it.
- If there is a disagreement between members, the outcome will be decided by a vote. At the moment, we have 4 people to a 3-1 vote will need to be achieved.
- We can meet either online on Discord or in the library on campus at a scheduled time.
- The repo owner (Dominic) is most likely responsible, but anyone can submit.

Team Roles:

Manager (Dominic)

- Facilitates team communication

- Helps and give guidance where needed

Camera Team (Dylan, Ryder)

- Plans the infrastructure (IDE/programming languages/ etc.) before development

- Builds the camera classes based on the design document

- Tests the camera's functionality independently of the backend

- Tests the camera's functionality regarding communication with the backend

Backend Team (Alex, Ryan)

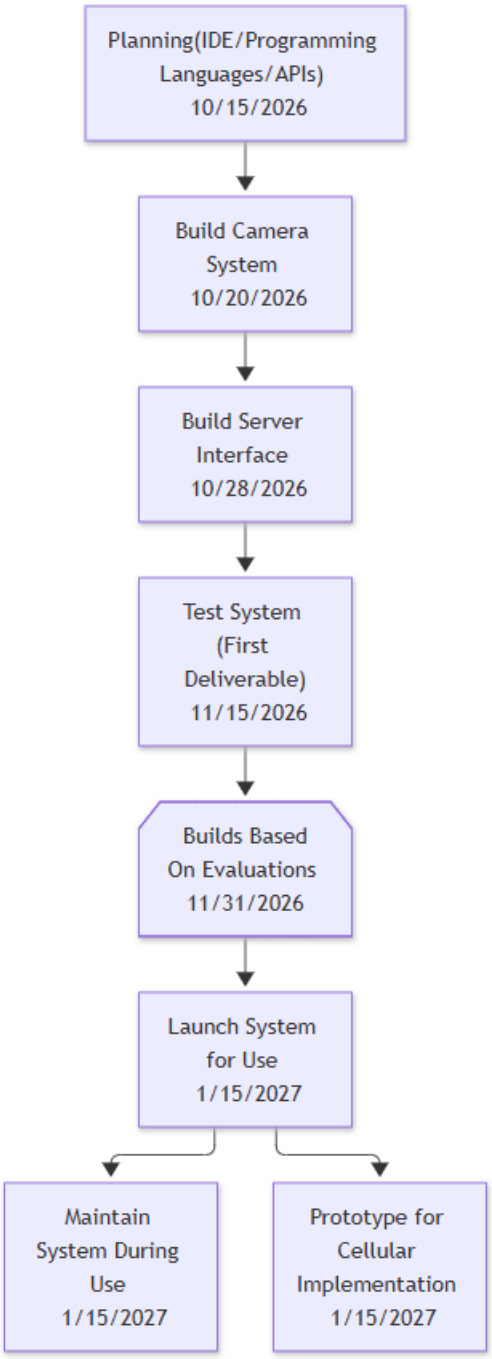
- Plans the infrastructure (IDE/programming languages/HTTP requests/ etc.) before development

- Builds the backend based on the design document

- Tests the backend before connecting with the camera

- Tests compatibility with the camera

Timeline:



Testing:

Data Cleanup Test Cases:

```
#region Data Cleanup
import time
from time import struct_time as Time
class ImgRecord:
  time: float
  uuid: str
  archived: bool
  pass


class db:
  @staticmethod
  def is_old_enough(rec: ImgRecord) -> bool:
    return True
  @staticmethod
  def add(rec: ImgRecord) -> None:
    pass
  OLD_OFFSET = 60*60*24*7*4
  @staticmethod
  def query(q: str|None = None) -> list[ImgRecord]:
    return []
  @staticmethod
  def cleanup():
```

```
    pass
#endregion

# Unit Test
"""
This unit test tests the helper function
is_old_enough on DB. This function should
return true if the record is older than one month
(threshold for removal.) Otherwise, it should
return false.

Full test set is all possible Image Records. This
set is impractical to cover, since it includes
variation other than in the time field
(the field we are examining within this tool).

Even within the time field, full testing is
impractical. Instead, we focus on very specific ages
of the records: over a month old, not a month old,
new, and future. Future records are erroneous
in nature, but should not result in a
malformed program state.

Our test tests one of each category, with an
```

```
additional test for being too old. If additional
tests are wanted for this feature, add them here.
"""
def has_correct_filter():
  record = ImgRecord()
  record.time = time.time()
  record.time -= 60 * 60 * 24 * 29
  assert db.is_old_enough(record)

  record = ImgRecord()
  record.time = time.time() - 42069
  assert not db.is_old_enough(record)

  rec2 = ImgRecord()
  rec2.time = time.time()
  assert not db.is_old_enough(rec2)

  record = ImgRecord()
  record.time = time.time() + 1000
  assert not db.is_old_enough(record)

  record = ImgRecord()
  record.time = time.time() - 67676767
  assert db.is_old_enough(record)
```

Integration Test

"""

The integration test checks general DB I/O, as well
as the automated record cleanup process db.cleanup().

Our possible test vectors are all possible database
states. Even more so than the unit test, the
set of test cases is entirely impossible to test
within a reasonable time.

We test three main categories: old, new, future, and
archived. Again, future is erroneous, but should not
cause a malformed database state. Old records
(older than a month) should be removed by
db.cleanup(), with some exceptions. Actual image
archival is not being tested here; simply that the
archived images will not be deleted for up to a
year.

This also checks the db to ensure that there
were no other records sneaking through. This
test thus requires a LOCK on the db while being
run.

```
"""
def finds_bad_entries():
  record = ImgRecord()
  record.time = time.time()
  db.add(record)
  db.cleanup()
  xs = db.query(f"UUID = {record.uuid}")
  assert record in xs

  # too old
  record = ImgRecord()
  record.time = time.time() - 60*60*24*29
  db.add(record)
  db.cleanup()
  xs = db.query(f"UUID = {record.uuid}")
  assert record not in xs

  # future
  record = ImgRecord()
  record.time = time.time() + 10000
  db.add(record)
  db.cleanup()
  xs = db.query(f"UUID = {record.uuid}")
  assert record in xs
```

```
# almost too old
record = ImgRecord()
record.time = time.time() - 60*60*24*27
db.add(record)
db.cleanup()
assert record in db.query(f"UUID = {record.uuid}")

# archived image stays
record = ImgRecord()
record.time = time.time() - 67676767
db.add(record)
db.cleanup()
assert record in db.query(f"UUID = {record.uuid}")

# old archived image goes
record = ImgRecord()
record.time = time.time() - 6767676767
db.add(record)
db.cleanup()
assert record not in db.query(f"UUID = {record.uuid}")

# no records that shouldn't be there are there
db.cleanup()
```

```
  for x in db.query():
    assert not db.is_old_enough(x)

# System Test
"""
System Test: This test is done in a live environment,
but due to the limited uptime this test server
is expected to have, cannot and should not
be automated.

This test is a full test of the archival service
only archiving a single image. Test vectors are
all real-world environments; somehow, this is
even more impossible to perform a full test set on.

Given the time required to run this test, this test
cannot be expected to be run multiple times
subsequently; thus, it should be run in parallel.

Additionally, to prevent any potential interference
from electronic failure, this test WILL NOT be run
automatically. It MUST be performed manually.

Recommended tools to run perform test include
```

a calendar and a notebook.

"""

"""

1. Camera submits photos over course of a day

2. Mark down UUIDs of all photos submitted that day at end

3. Wait 5 weeks

4. Confirm only one of those photos is in DB

5. Wait 1 year

6. Confirm none of these photos exists in DB

"""

Photo System Test Cases:

Unit

"""

Every image must carry a time and location so the evaluation system and users know when and where changes in bluff were observed. These unit tests call get_time() and get_geolocation(), checking both functions to confirm that it returns a value instead of null. The expected output is a non-null value, with the tests failing if a time or location is unavailable or null.

"""

Get a timestamp

time: TimeStamp = ImageCapture.get_time()

```
# Ensure you got a timestamp
if time is null:
        return FAIL

return PASS

# Get geolocation
geolocation: GeoLocation = ImageCapture.get_geolocation()

# Ensure you got geolocation
if geolocation is null:
        return FAIL

return PASS
```

Integration

"""

This integration test checks the ability of a camera to properly take a photo and remotely send an image to the server's CameraInterface. The test begins right after ImageCaptures take_picture() is called, which tests if the image has properly been captured. After this process, CameraInterface tests if the image sent after post_image() call has been properly received.

The expected output is that the image appears in the database within the timer. It fails if the image returns null, post_image() errors, or he image never appears.

These tests are necessary to ensure that the functions in ImageCapture and CameraInterface properly work across WiFi, with any errors in both the taking and the sending of a photo getting caught, which ensures that the image is captured successfully and sent through the database to warn of any potential changes.
"""

```
# Testing ImageCapture functions
# Capture an image
image: Image = ImageCapture.take_picture()

# Ensure the image was captured
if image is null:
    return FAIL

# Send the image to the CameraInterface
ImageCapture.post_image(image)

# Testing CameraInterface functions
# Wait for the image to be processed
time.sleep(100)

# Ensure the CameraInterface received an image by checking for it in the database
if EvaluationSystem.get_image(image) is null:
    return FAIL
```

return PASS

System

What we are testing: The workflow from the camera taking a picture all the way to sounding the alarm. We are ensuring that if the camera takes a picture with an alarming state to the bluff, the alarm sounds.

The camera takes a picture of the bluff in a bad condition (Or we feed it an alarming image). The camera then sends the picture over wifi to the camera interface in the server. The interface sends the image to the evaluation system. The evaluation system sounds the alarm and sends an alert to all subscribed.

Config Test Cases:

Unit Tests:

"""

This unit test tests two components of update_resolution() in CameraConfig. The first test checks the lower bounds of any width and height inputted to ensure that no measurement is less than or equal to zero. The second test then checks if the resolution measurements that were stored into CameraConfig copied over successfully.

The first test is necessary in order to first check if the input provided by the user is possible to implement for the resolution, as zero and lower could error the image capture software when inputted into the resolution. The second test is also necessary to confirm that the resolution config has been copied over successfully.

The expected output is a resolution that equals the valid input provided. A test fails if an invalid resolution is inputted, or a value was stored incorrectly.

"""

```
# Making test values
test_width: int = 10
test_height: int = 10

CameraConfig.update_resolution(test_width, test_height)

# Test bounds
if CameraConfig.width <= 0 or CameraConfig.height <= 0:
        return FAIL

# Test if values were copied over correctly
if CameraConfig.width != test_width or CameraConfig.height != test_height:
        return FAIL

return PASS
```

"""

This unit test tests two components of update_interval() in CameraConfig. The first test checks the lower bounds of any int inputted to ensure that no measurement is less than or equal to zero. The second test then checks if the interval that was stored into CameraConfig copied over successfully.

The first test is necessary in order to first check if the input provided by the user is possible to implement for the interval, as zero and lower could error the image capture software when inputted into the config. The second test is also necessary to confirm that the interval has been copied over successfully.

The expected output is an interval that equals the valid input provided. A test fails if an invalid interval is inputted, or a value was stored incorrectly.

"""

```
# Test interval to input into the config
test_interval: int = 60

CameraConfig.update_interval(test_interval)

# Test if interval is less than or equal to zero
if CameraConfig.interval <= 0:
    return FAIL

# Test if interval was copied over correctly
if CameraConfig.interval != test_interval:
        return FAIL

return PASS
```

"""

This unit test tests one component of set_color() in CameraConfig. The test checks if the Boolean provided by the user in set_color() was updated correctly in CameraConfig. This test is necessary to ensure that the boolean was properly updated, as the wrong setting would cause the image capture software to improperly take photos with the wrong filter on.

The expected output is the stored Boolean equaling the input. The test fails if a Boolean setting does not change after input.

"""

```
# First test, checking if true
CameraConfig.set_color(true)
# Test if color was updated correctly
for CameraConfig.color != true:
    return FAIL

# Second test, checking if false
CameraConfig.set_color(false)
# Test if color was updated correctly
for CameraConfig.color != false:
    return FAIL

return PASS
```

"""

This unit test tests one component of set_night_vision() in CameraConfig. The test checks if the Boolean provided by the user in set_night_vision() was updated correctly in CameraConfig. This test is necessary to ensure that the boolean was properly updated, as night vision enabled/disabled at a certain hour would cause images captured to be difficult to process.

The expected output is the stored Boolean equaling the input. The test fails if a Boolean setting does not change after input.

"""

```
# First test, checking if true
CameraConfig.set_night_vision(true)
# Test if night vision is updated correctly
if CameraConfig.night_vision != true:
    return FAIL

# Second test, checking if false
CameraConfig.set_night_vision(false)
# Test if night vision is updated correctly
if CameraConfig.night_vision != false:
    return FAIL

return PASS
```

Integration

"""

This integration test checks the ability of a camera's config to remotely obtain updated configurations from the server's CameraConfig. We test all four configurations located in both configs: resolution, color, night_vision, and interval.

Once any of the four variables in CameraConfig on server are updated, camera's Config will then remotely update its values with user values. After this process is run, it compares its values against user values to ensure that they've properly copied over.

The expected output is that Config returns exactly the values set on the server. It fails if any value is missing or incorrect from the user-inputted values.

This is important to ensure that Config hasn't improperly stored any values that aren't what the user intends, keeping all cameras consistent with the settings inputted by user.

"""

On Server Side

Create test variables

test_resolution: Array[int] = [100, 100]

test_color: bool = false

test_night_vision: bool = false

```
test_interval: int = 10

# Set the values in CameraConfig
CameraConfig.update_resolution(test_resolution)

CameraConfig.set_color(test_color)

CameraConfig.set_night_vision(test_night_vision)

CameraConfig.update_interval(interval)



## On Camera Side
# Ensure the camera can access the new values
if Config.get_resolution() != test_resolution:
        return FAIL

if Config.get_color() != test_color:
        return FAIL

if Config.get_interval() != test_interval:
        return FAIL

if Config.get_night_vision() != test_night_vision:
```

return FALSE

return PASS

System

A user updates the values in CameraConfig on the server. Then, once the camera takes a picture, they are able to view the picture and verify that the picture was taken with the new config settings.

DB Management Strategy

For the system, we chose to maintain two databases. This decision comes with a number of consequences. The primary candidate database management strategy will be covered most in depth, but other systems will be mentioned where they were brought up during development process.

Candidate: Multi-Database System

This system design involves two separate databases working together: a SQL database, and a NoSQL database. SQL databases are fast to query but can get bloated and might be hard to store thousands of records on separate drives in bulk. However, SQL's benefits of rapid indexing are difficult to ignore.

We decided to use two databases. The first, a SQL database, will serve as the indexer for the other, a NoSQL database. After some consideration, it was determined that a KV database which stores its values on disk would be a good match for this purpose.

SQL Database

The SQL database should be able to index by whatever is necessary to provide the best results quickly. We have not selected a particular SQL database software for this purpose, as it's unclear which one fits the single-table many-record case we have here.

The schema for the indexer should be able to be indexed by: the camera that took the photo, where the photo was taken, the time it was taken at, the location where the image is stored, and the position the image was taken from. One value in that table should directly map to the image in the NoSQL database.

NoSQL Database

For the KV database, we selected OpenRiak. While OpenRiak is intended more for rapid readwrite applications, it still fits the design requirements for image review and retrieval at a larger scale, should the client wish to scale up operations. This database should remain the authoritative database of the two. If something has gone wrong (catastrophically or not) with the SQL indexer, it can be reconstructed by iterating the values in the OpenRiak, reading their EXIF data, and rebuilding the metadata table.

The leveled backend for OpenRiak meets these requirements best. Values (images) will not be stored in-memory but will be stored on the disk. Additionally, values are compressed automatically by the backend; this assists in minimizing the data storage overhead and could perhaps permit additional nodes in the local ring to exist. If any images are provided with an unexpectedly large file size, OpenRiak should not fail to store these images. This means that camera upgrades or changes can be performed without needing to tinker with the database configuration, as long as the data can be provided to the processor.

Redis

Redis was rejected for this application's use case. The largest factor in this decision was that Redis is an in-memory database. Keeping the values of this database in-memory is a poor idea, given that each value can be estimated to be 10MB in size.

Helper

Given that we will be using two databases, it's clear that a database helper will be required to perform some operations. Initial review suggests the following functionality as clearly required:

- **Record addition.** A key must be generated for the image to be stored, which is then provided to the indexer (along with image metadata) and stored. This key should also be provided to the OpenRiak database and used to store the given image.
- **Indexer population.** In the event of any failure to populate the indexer table or database, the helper should be able to populate and update it with the required records.

- **Image store cleanup.** While OpenRiak may make storing the image files potentially more efficient, storage space is limited. A process to clean up any old images that should not be stored any longer must be implemented.
- **Manual record deletion.** Given that the OpenRiak KV database is the authority, deletion MUST be performed there. The indexer can be used to determine which records should and should not be deleted.
- **Rollbacks.** If a database rollback is required, this must be performed by OpenRiak. If there is a time-proximal SQL database backup, this can be rolled back to and then populated from the image store; otherwise, full repopulation may be necessary.

Analysis

This scenario takes the best advantage of both the KV-model and an SQL database. SQL databases can be indexed to search for any relevant data quickly, while KV-model databases can serve individual values at a similar speed. OpenRiak compressing the values it stores means that less disk space can be expected to be consumed, and its data storage model means that, if scaled, the database becomes more reliable. This has additional applications; if a read-only interface for the OpenRiak database is exposed to the internet, sending alerts to the smart devices of operators/inspectors/authorities can be done without needing to send potentially multiple 10MB images to each of them. Instead, a copy of the report, along with a

key for the image, can be provided to each user.

However, additional complexity comes from using two separate database systems. If either database goes down for an extended period, the other is functionally useless. Whenever a record is deleted in the SQL database, on its own, this does not immediately delete the record in the OpenRiak database. However, this also means that if any record is improperly deleted, because the indexer entry can be reconstructed entirely from EXIF metadata, the indexer database itself can recover from only the OpenRiak database.

A database helper will be required to manage this complexity. While the theoretical complexity of this helper is reasonably low and would be required regardless of the presence of the two-database solution, its complexity is increased by using two different databases.

+ Prevents SQL database lockups from large single-read or write operations.
+ Catastrophic SQL database failure can be recovered relatively easily. + Can scale larger with project expansion in the future with little demand beyond storage upon the database. - Added complexity from

needing to coordinate the contents of both databases. - Compromise of OpenRiak database is effectively composed of both databases.

SQL Only

Using a single-table SQL database could also work for this functionality. This would encourage the use of a single-table model. While this would make the database quite simple, it would also create a single point of failure.

Additionally, if images are ever provided that do not conform to the maximum specified file size, the model will entirely fail to store them. Given the safety consequences for a failure to store files (as these could not be compared to previous files), this should be avoided where possible.
Using a VARBINARY field with a large maximum size would help resolve this issue.

NoSQL Only

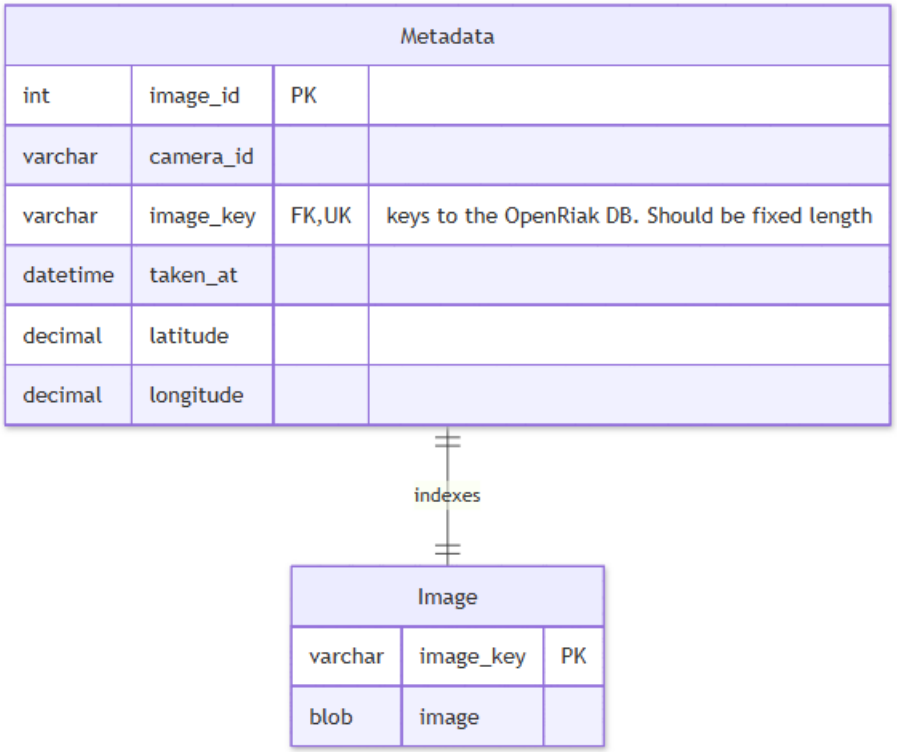
There is no strong reason to use a NoSQL-only database structure for this project. Database entries must be cleaned up over time.

A KV-model database could work but would require checking the EXIF metadata of all files to perform said cleanup. While this is possible, it's far from desirable.

A document-model database faces similar issues. Document-model databases and KV-model databases are similar in structure, in that all elements are accessed by key, but document model databases return additional data. While this would relieve us of the need to deserialize EXIF whenever we needed to read metadata, it still requires some amount of the file to be served whenever it is checked. In isolation, it is not necessarily sorted, either, meaning that looking for images in a particular date range would require collecting all images.

Graph-model and column-model databases are NoSQL for this data model and were therefore not considered.

Database Structure Diagram:



Further Context/Analysis on DB Structure:

For data management, the information will be split across two databases. One database (SQL) stores metadata for each image, while the other (NoSQL) stores the images themselves. We did this due to the large file sizes of each image. The databases will be run in parallel, and image data can be accessed through a UUID, which is consistent across both databases. The UUID will be generated for an image when the request to add an image to the databases starts.

SQL Indexed Table

Metadata on an image will be stored in an SQL table. Every row will be information pertaining to a specific image. The image metadata is searchable by the image's UUID. A single row will hold: The UUID of an image, the image's timestamp, and the image's geolocation. This allows an image's UUID to be looked up by its timestamp, a timestamp to be looked up by its UUID, or any other combination.

NoSQL Riak Database

The NoSQL Database will store the image files themselves. Like the SQL Database, you will be able to look up an image based on its UUID. The UUID will map to the image, which will be stored on the system's disc. This way, we can store a large amount of data and easily access it. It was decided that this would not be plausible in a single SQL database due to the large image file sizes. Riak was specifically chosen because it can store UUIDs in memory while storing image files on disc.